

TAREA 2.1 - Explorando nuevos widgets de PySide 6



PyQt6

ÍNDICE

1. QRadioButton	2
2. QTabWidget	5
3. QProgressBar	6
4. QDateTimeEdit	9
5. QSlider	11
6. QDial	12

1. QRadioButton

- **Descripción general: qué es y para qué se usa.**

Es un botón de selección, que nos permite de una serie de opciones elegir solamente una de todas.

Se suele usar normalmente para formularios (*género, método de pago, talla de ropa*), encuestas y algunas opciones en configuraciones, etc...

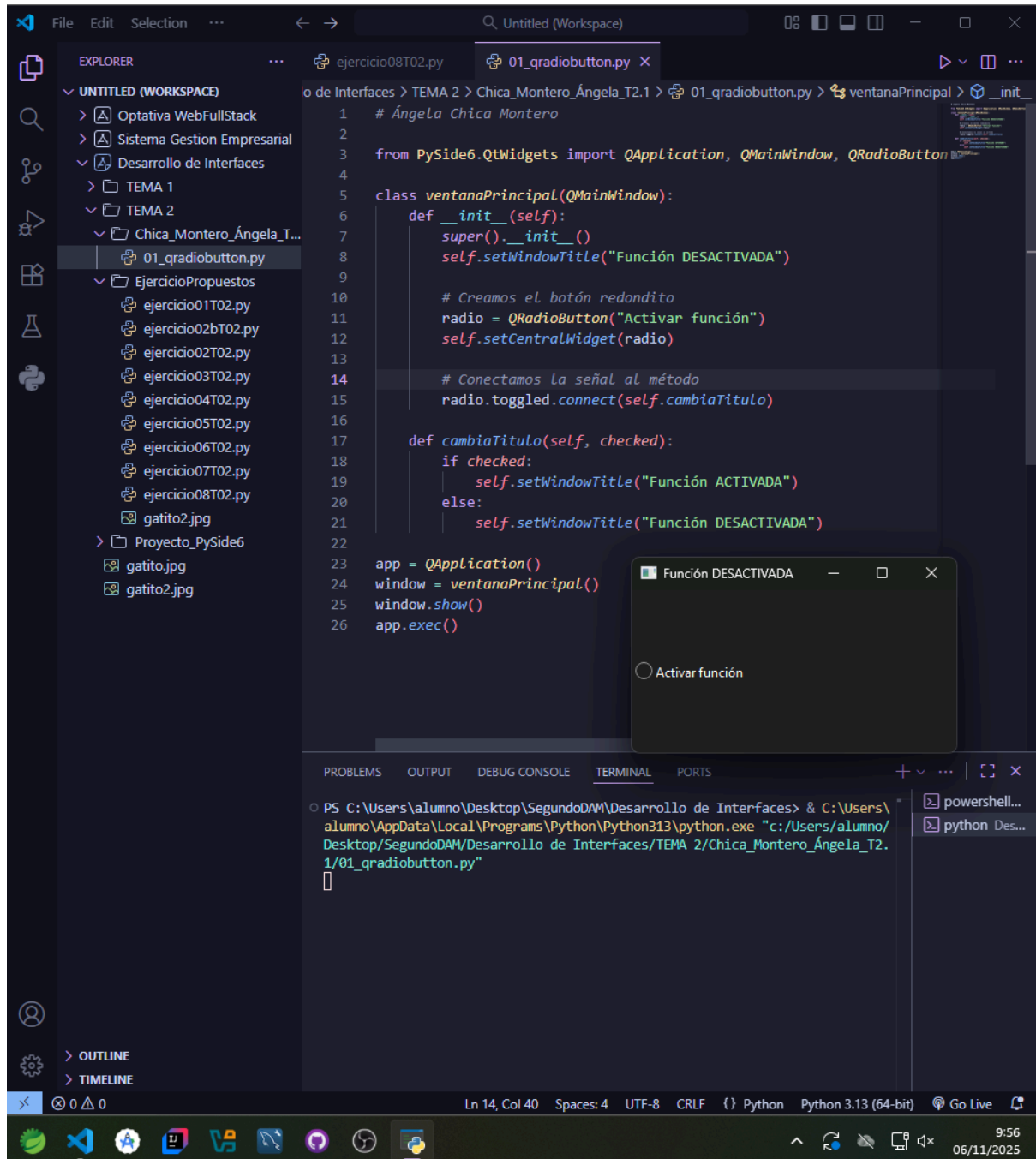
- **Principales métodos (mínimo los de la tabla anterior) y su descripción.**

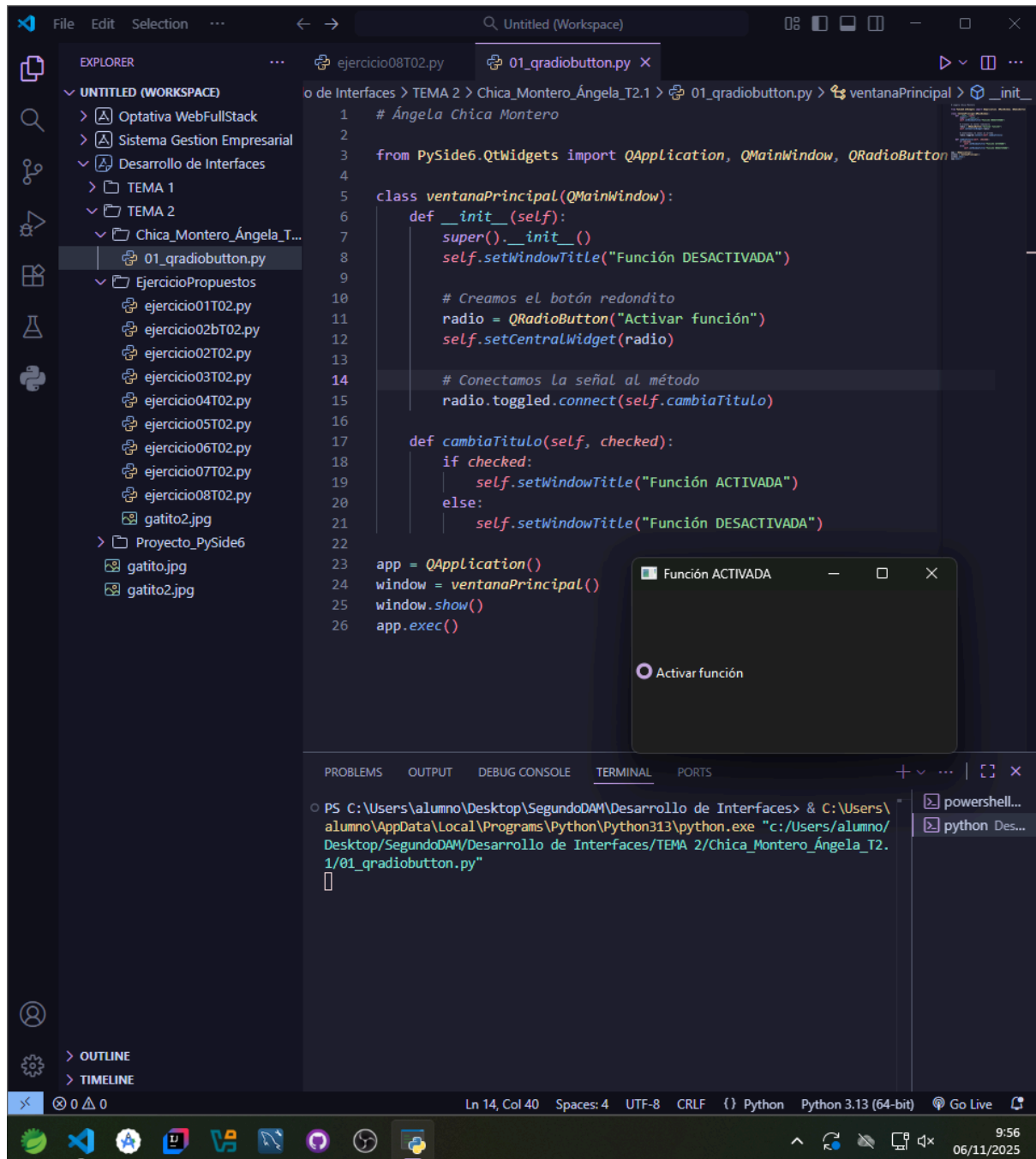
- **.setChecked(bool):** al ser booleano sirve para cambiar el estado del botón → *true: marcado / false → desmarcado*
- **.isChecked():** parecido al anterior pero devuelve true si se ha seleccionado o false si es al contrario
- **.text():** recupera el texto escrito del botón
- **.setText(str text):** podemos establecer el texto que deseemos en el botón
- **.setCheckable(bool checkable):** decidimos si el botón podemos marcarlo (*true*) o no (*false*)
- **minimumSizeHint():** elegimos el tamaño que queremos poner al botón y así no puede redimensionarse menos de lo que pusimos

- **Señales (slots) más frecuentes y su uso.**

- **.toggled():** lo usamos para mostrar el cambio que se ha tenido el botón
- **.clicked():** como el anterior pero en este caso solo es cuando hacemos click en él
- **.pressed():** para guardar lo que ocurre al presionar el botón o lo que queramos que haga
- **.released():** igual que el anterior pero en este caso sería al deseleccionarlo

- **Captura de pantalla de la ejecución del ejercicio propuesto que muestre su funcionamiento.**





- **WebGrafía:**

<https://doc.qt.io/qt-6/qradiobutton.html>

<https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/index.html>

<https://hektorprofe.github.io/qt-pyside/conceptos-basicos/senales-receptos-si-gnals-slots/>

2. QTabWidget

- **Descripción general: qué es y para qué se usa.**

Se utiliza para crear una interfaz con pestañas, donde cada pestaña tiene su propio conjunto de widgets.

- **Principales métodos (mínimo los de la tabla anterior) y su descripción.**

- **.addTab(widget, "Título"):** con este método añadimos una nueva pestaña al QTabWidget. Primero le pasamos el tipo de contenido que va a mostrar y luego lo que debe mostrar

Ejemplo (sacado del ejercicio siguiente):

```
self.pestanas.addTab(ventanal, "Pestaña 1")
```

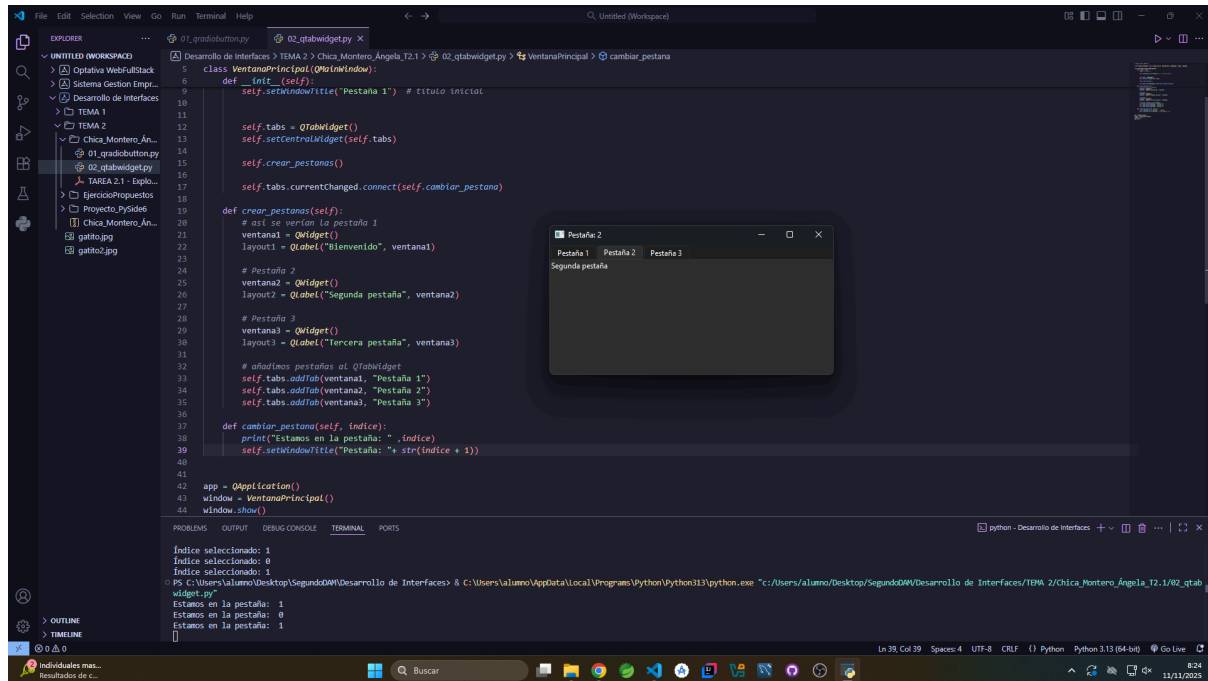
- **.setCurrentIndex(int):** como su propia traducción indica, podemos cambiar la pestaña de donde estamos con el índice que digamos
- **.count():** muestra el total de pestañas que tengamos
- **.iconSize():** nos devuelve el tamaño de los iconos usados en las pestañas
- **.insertTab():** añadimos una nueva pestaña en la posición que deseemos moviendo las demás.

- **Señales (slots) más frecuentes y su uso.**

- **.currentChanged(int):** cuando cambiamos de pestaña nos mostrará según su índice donde estamos actualmente
- **.setCurrentIndex():** se usa para poder cambiar a una pestaña directamente cuando lo conectemos a una acción en concreto (*pulsar un botón nos manda a la ventana 4*)

- **Captura de pantalla de la ejecución del ejercicio propuesto que muestre su funcionamiento.**

En la siguiente captura de pantalla se puede ver el funcionamiento del código del ejercicio propuesto:



- **WebGrafía**

<https://doc.qt.io/qt-6/qtabwidget.html>

3. QProgressBar

- **Descripción general: qué es y para qué se usa.**

Se usa para mostrar visualmente el progreso de una tarea o su avance dentro de un progreso definido (*del 0 al 100, 0 al 20, ...*). Se usan para mostrar el progreso de una descarga o copia de archivos de uno a otro, ...

- **Principales métodos (mínimo los de la tabla anterior) y su descripción.**

- **.setValue(int):**

Establece el valor actual de la barra de progreso dentro del rango definido.

- **.setRange(min, max)**

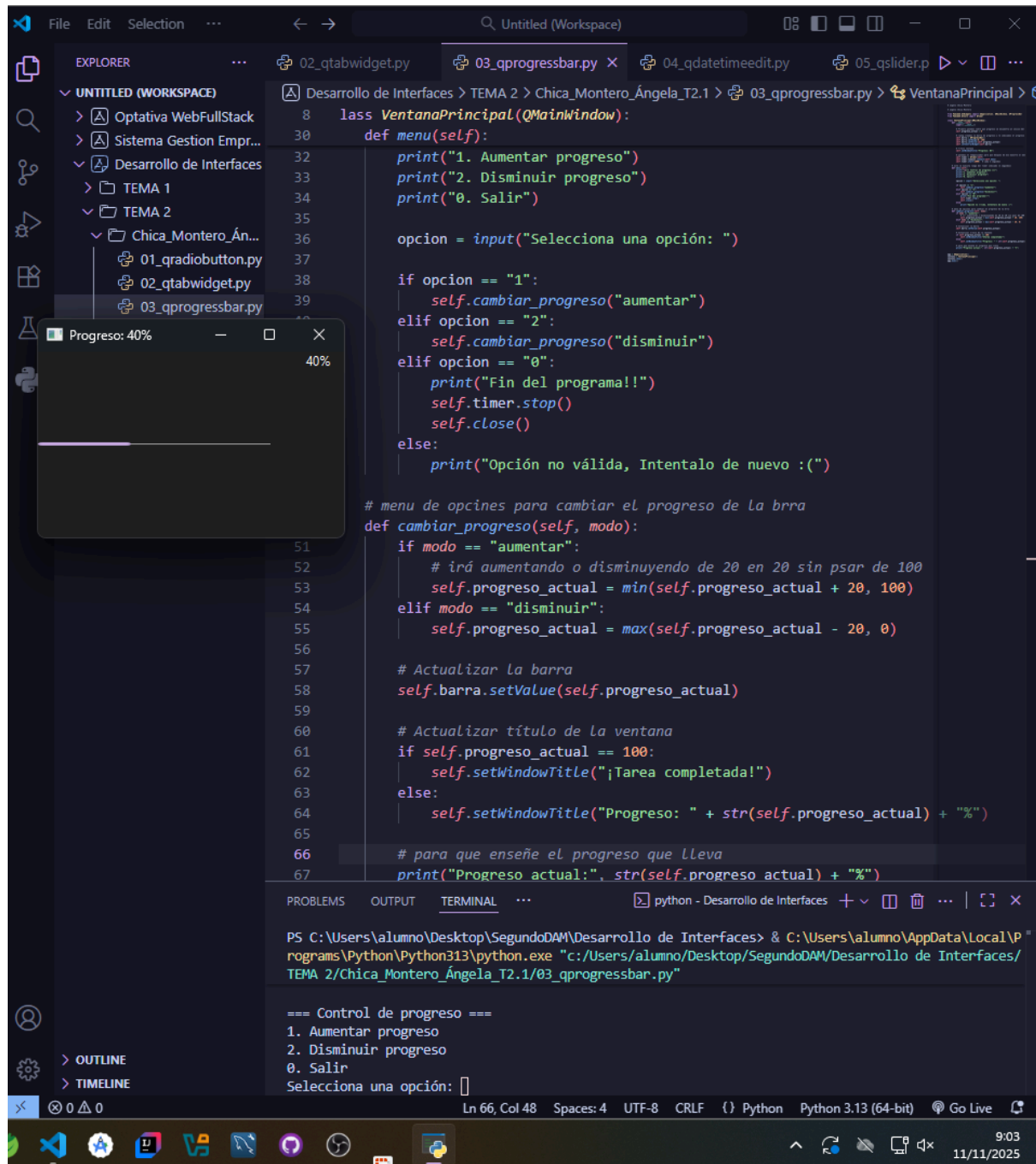
Define el rango de valores permitido para la barra. Por defecto es de 0 a 100, podemos poner el que queramos

- **.reset()**
Reinicia la barra al valor mínimo del rango, volviendo a su estado inicial.
- **.value():**
Devuelve el valor actual de la barra de progreso.
- **.setMinimum(int) / .setMaximum(int)**
Permiten definir los límites inferior y superior del rango por separado.
- **.textVisible(bool)**
Permite mostrar u ocultar el texto (el porcentaje) dentro de la barra.

- **Señales (slots) más frecuentes y su uso.**

Como se indica en los apuntes de la tarea no tiene slots propio, pero usa

- **Captura de pantalla de la ejecución del ejercicio propuesto que muestre su funcionamiento.**



- WebGrafía

<https://doc.qt.io/archives/qtforpython-5/PySide2/QtWidgets/QProgressBar.html>

<https://www.pythontutorial.net/pyqt/pyqt-qprogressbar/>

4. QDateTimeEdit

- **Descripción general: qué es y para qué se usa.**

Básicamente, este widget nos permite crear una ventana donde elegir la fecha exacta y hora que deseemos

- **Principales métodos (mínimo los de la tabla anterior) y su descripción.**

- **.setDateTime(QDateTime)**
Establece la fecha y hora inicial del widget.
- **.dateTime()**
Devuelve la fecha y hora seleccionadas actualmente.
- **.setDisplayFormat(str)**
Define el formato de visualización, por ejemplo "dddd, d 'de' MMMM 'de' yyyy hh:mm".
- **.setDate(QDate)**
Cambia solo la fecha del selector.
- **.setTime(QTime)**
Cambia solo la hora del selector.
- **.date()**
Devuelve solo la fecha actual seleccionada (QDate).
- **.time()**
Devuelve solo la hora actual seleccionada (QTime).

- **Señales (slots) más frecuentes y su uso.**

No encontré más slots que los indicado:

- **dateChanged(QDate)**
Lo usamos cuando queremos cambiar solo la fecha y mostrarlo.
- **timeChanged(QTime)**
Igual que el anterior pero solo con la hora.
- **dateTimeChanged(QDateTime)**
Es como los otros dos pero los usa a la vez, mostrando el cambio de la fecha y hora.

- **Captura de pantalla de la ejecución del ejercicio propuesto que muestre su funcionamiento.**

Para realizar el código usé principalmente el segundo enlace de la webgrafía que lo explica todo muy bien sinceramente.

The screenshot shows a Visual Studio Code editor with a Python script named `04_qdatetimeedit.py` open. The script defines a `VentanaPrincipal` class that inherits from `QMainWindow`. It includes a date and time selector widget, sets the display format to Spanish, and updates the window title based on the selected date and time. The terminal output shows the execution of the script, displaying the selected date and time in Spanish.

```

1 # Ángela Chica Montero
2
3 from PySide6.QtWidgets import QApplication, QMainWindow, QDateTimeEdit
4 from PySide6.QtCore import QDateTime
5
6 class VentanaPrincipal(QMainWindow):
7     def __init__(self):
8         super().__init__()
9
10        self.selector_fecha_hora = QDateTimeEdit()
11        self.setCentralWidget(self.selector_fecha_hora)
12
13        # con esto muestra el día y hora de hoy
14        self.selector_fecha_hora.setDateTime(QDateTime.currentDateTime())
15
16        # Lo ponemos en formato español porque por defecto sería en inglés con el año por delante
17        self.selector_fecha_hora.setDisplayFormat("dddd, d 'de' MMMM 'de' yyyy hh:mm")
18
19        # para cambiar el título según la fecha que escogamos
20        self.selector_fecha_hora.dateTimeChanged.connect(self.actualizar_titulo)
21
22        # igual que el anterior pero el título será primero la fecha inicial
23        self.actualizar_titulo(self.selector_fecha_hora.dateTime())
24
25        # cambia el título al cambiar la hora y mostramos cual es por consola
26        # lo he puesto en formato español con el día por delante
27        def actualizar_titulo(self, fecha_hora):
28
29            print("Fecha elegida:", fecha_hora.toString("dddd, d 'de' MMMM 'de' yyyy hh:mm"))
30
31            self.setWindowTitle(fecha_hora.toString("dddd, d 'de' MMMM 'de' yyyy hh:mm"))
32
33
34 app = QApplication([])
35 ventana = VentanaPrincipal()
36 ventana.show()
37 app.exec()
38

```

The terminal output shows the execution of the script, displaying the selected date and time in Spanish:

```

PS C:\Users\alumno\Desktop\SegundoDAM\Desarrollo de Interfaces> & C:\Users\alumno\AppData\Local\Programs\Python\Python313\pyt
hon.exe "c:/Users/alumno/Desktop/SegundoDAM/Desarrollo de Interfaces/TEMA 2/Chica_Montero_Ángela_T2.1/04_qdatetimeedit.py"
Fecha elegida: Tuesday, 11 de November de 2025 09:24
Fecha elegida: Wednesday, 12 de November de 2025 09:24
Fecha elegida: Thursday, 13 de November de 2025 09:24

```

- **WebGrafía**

<https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QDateTimeEdit.html>

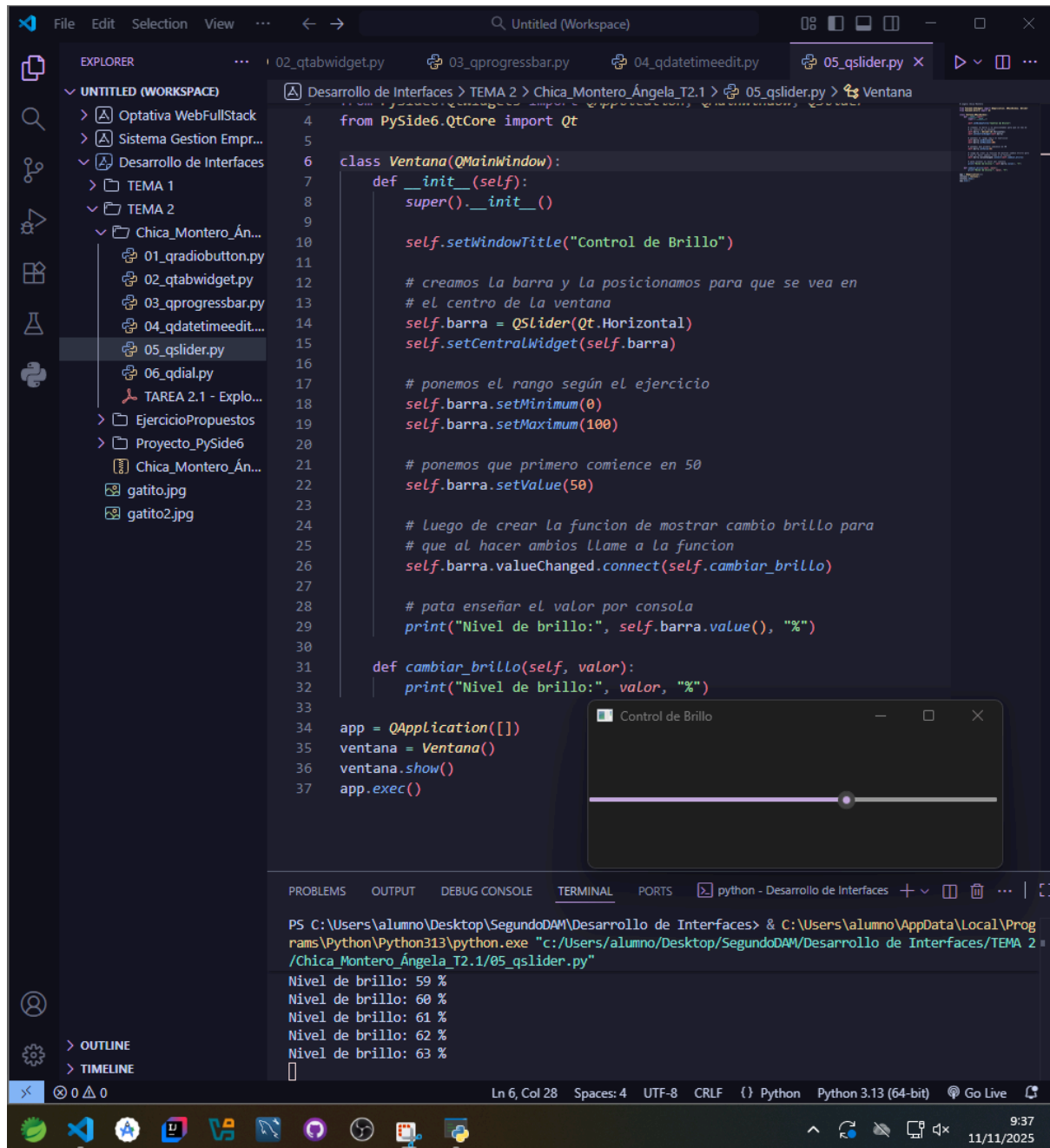
<https://www.pythontutorial.net/pyqt/pyqt-qdatetimeedit/>

5. QSlider

- **Descripción general: qué es y para qué se usa.**

Es parecido a la barra de progreso pero en este caso nos permite seleccionar un valor numérico dentro de un rango usando una barra deslizante. Se suele usar para controlar el volumen, en un capítulo de una serie elegir el minuto correcto, etc ...

- **Principales métodos (mínimo los de la tabla anterior) y su descripción.**
 - **.setMinimum(int)**
Establece el valor mínimo que puede tomar la barra.
 - **.setMaximum(int)**
Igual que el anterior pero en este caso en máximo.
 - **.setValue(int)**
Establece el valor actual que toma la barra.
 - **.value()**
Devuelve el valor que tiene en ese momento la barra.
 - **.setOrientation(Qt.Horizontal/Qt.Vertical)**
Cambia la orientación de la barra (horizontal o vertical).
- **Señales (slots) más frecuentes y su uso.**
 - **.valueChanged(int)**
Devuelve el valor cuando cambia la barra
 - **.sliderMoved(int)**
Parecido al anterior pero es solo cuando es el usuario quien mueve el valor de la barra
 - **.sliderPressed()**
Al presionar el deslizador
 - **.sliderReleased()**
Igual que el anterior pero al dejar de presionarlo
- **Captura de pantalla de la ejecución del ejercicio propuesto que muestre su funcionamiento.**



- **WebGrafía**

<https://doc.qt.io/qt-6/qslider.html>

<https://runebook.dev/es/docs/qt/qslider>

<https://acodigo.blogspot.com/2017/08/qcombobox-y-qslider.html>

6. QDial

- **Descripción general: qué es y para qué se usa.**

Es similar al qslider pero con forma circular, se suele usar para ajustar el volumen, cambiar el brillo, etc...

- **Principales métodos (mínimo los de la tabla anterior) y su descripción.**

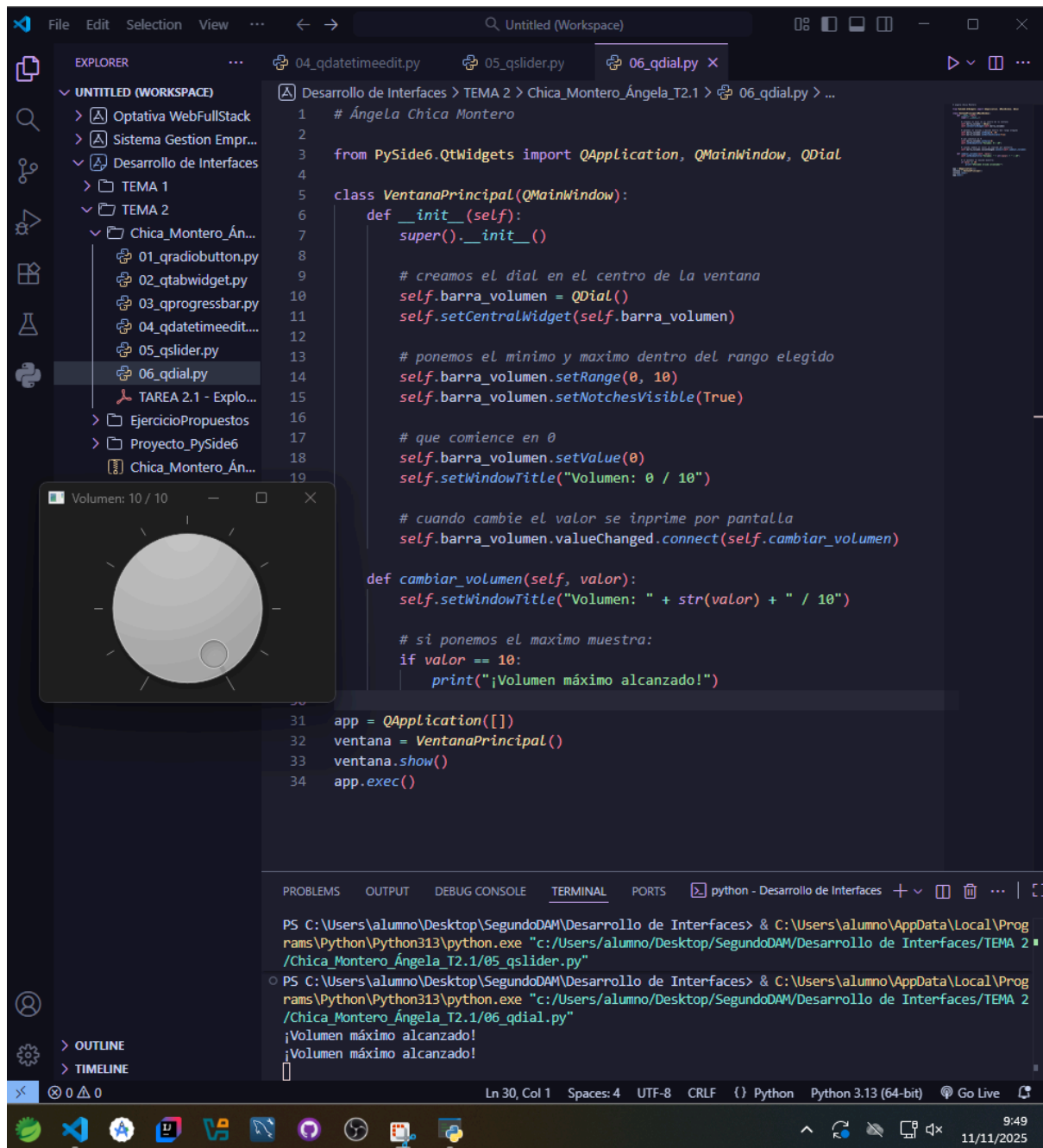
- **.setNotchesVisible(True/False)**
Muestra u oculta las marcas del propio Dial
- **.setRange(min, max)**
Nos permite establecer el rango de valores
- **.setMinimum(int)**
Ponemos el mínimo
- **.setMaximum(int)**
Ponemos el máximo
- **.setValue(int)**
Elegimos por defecto que valor tendrá el dial
- **.value()**
Devuelve el valor actual del dial.

- **Señales (slots) más frecuentes y su uso.**

Son parecidos al widget anterior:

- **.valueChanged(int)**
Se emite cada vez que cambia el valor, ya sea girando el dial o programáticamente.
- **.sliderMoved(int)**
Se emite cuando el usuario mueve manualmente el selector
- **.sliderPressed()**
Se emite al presionar la perilla.
- **.sliderReleased()**
Se emite al soltar la perilla después de moverla.

- **Captura de pantalla de la ejecución del ejercicio propuesto que muestre su funcionamiento.**



- **WebGrafía**

<https://doc.qt.io/qt-6/qdial.html>

<https://pythonbasics.org/qdial/>